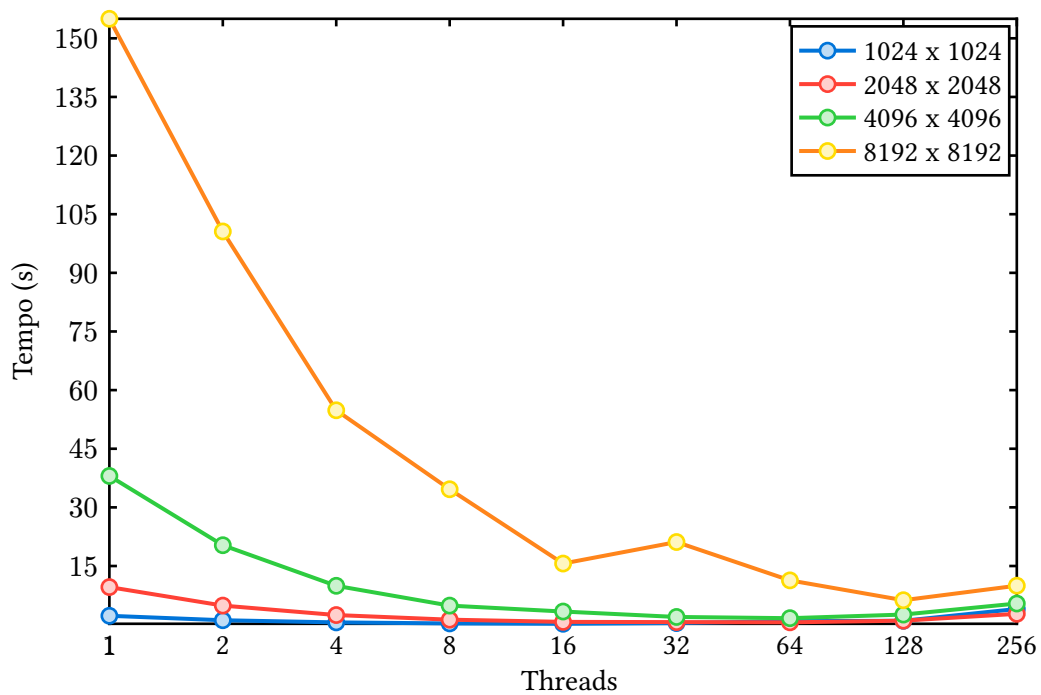


Análise de escalabilidade do problema Navier-Stokes

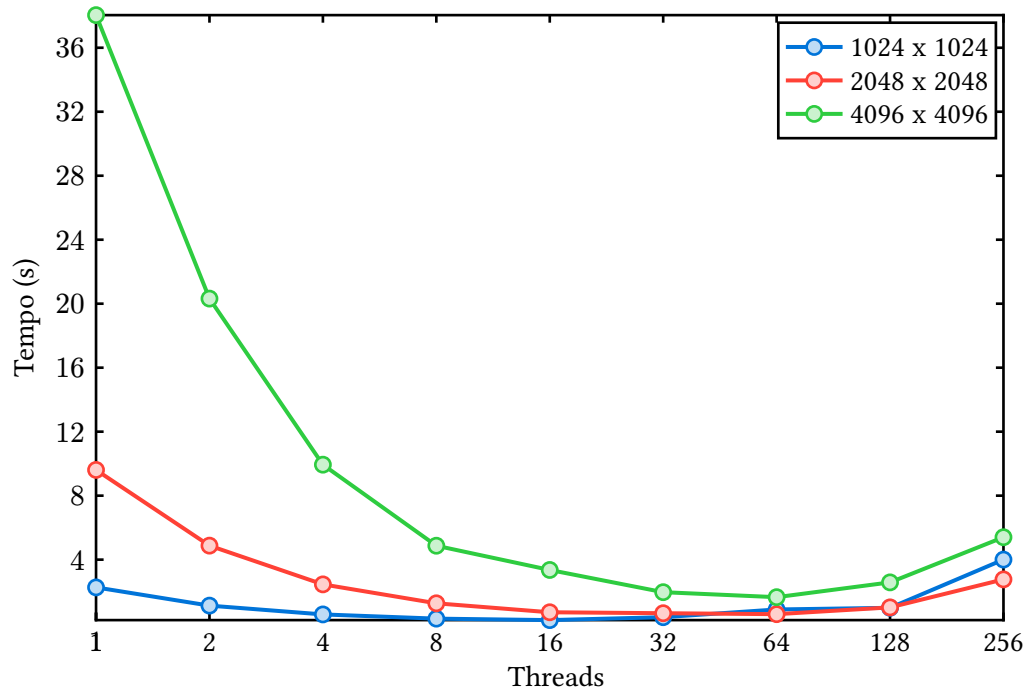
Para essa atividade, utilizei o nó de computação do NPAD amd-512 para realizar os testes. O código utilizado foi basicamente igual ao da atividade 11, com a diferença que a quantidade de threads foi definida dinamicamente dentro do programa em vez de ser fixo. Dessa forma apenas uma execução no NPAD foi o suficiente para fazer uma análise da escalabilidade do programa.

Para os testes, foi variado o tamanho do grid, variando entre 1024x1024 até 8192, e a quantidade de threads também foi variado entre 1 e 256 (total do amd-512).

I. Todos os dados



II. Sem o Outlier (8192x8192)



III. Anexo

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define NT 5000
#define DX 0.1f
#define DY 0.1f
#define DT 0.001f
#define VISC 0.1f

typedef struct {
    int nx;
    int ny;
    float **data;
} Grid2D;

Grid2D alocar_grid(int nx, int ny, float valor_base) {
    Grid2D grid = {
        .nx = nx,
        .ny = ny,
        .data = (float **)malloc(nx * sizeof(float *)),
    };

    for (int i = 0; i < nx; i++) {
        grid.data[i] = (float *)malloc(ny * sizeof(float));
    }

    for (int i = 0; i < grid.nx; i++) {
        for (int j = 0; j < grid.ny; j++) {
            grid.data[i][j] = valor_base;
        }
    }
}
```

```

    }
}
// Perturbação central
int cx = grid.nx / 2, cy = grid.ny / 2, r = 4;
for (int i = cx - r; i <= cx + r; i++) {
    for (int j = cy - r; j <= cy + r; j++) {
        grid.data[i][j] = 10.0f;
    }
}

return grid;
}

void calcular_proximo_passo(const Grid2D u_current, const Grid2D u_next,
                           const float nu, const float dt, const float dx,
                           const float dy) {
    const int nx = u_current.nx;
    const int ny = u_current.ny;
    const float idx2 = 1.0f / (dx * dx);
    const float idy2 = 1.0f / (dy * dy);

#pragma omp parallel for
    for (int i = 1; i < nx - 1; i++) {
        for (int j = 1; j < ny - 1; j++) {
            const float laplaciano =
                (u_current.data[i + 1][j] - 2.0f * u_current.data[i][j] +
                 u_current.data[i - 1][j]) *
                idx2 +
                (u_current.data[i][j + 1] - 2.0f * u_current.data[i][j] +
                 u_current.data[i][j - 1]) *
                idy2;

            u_next.data[i][j] = u_current.data[i][j] + dt * nu * laplaciano;
        }
    }
}

void salvar_metricas_csv(const char *nome_arquivo, int nx, int ny, int passos,
                        double tempo, int threads) {
    FILE *teste_existencia = fopen(nome_arquivo, "r");
    int precisa_cabecalho = (teste_existencia == NULL);
    if (teste_existencia != NULL) {
        fclose(teste_existencia);
    }

    FILE *f = fopen(nome_arquivo, "a");
    if (f == NULL) {
        fprintf(stderr, "Erro ao abrir o arquivo CSV para escrita.\n");
        return;
    }

    if (precisa_cabecalho) {
        fprintf(f, "tipo_execucao,nx,ny,passos,tempo,tempo_segundos,threads\n");
    }
}

```

```

    fprintf(f, "%d,%d,%d,%f,%d\n", nx, ny, passos, tempo, threads);

    fclose(f);
}

void liberar_grid(Grid2D *grid) {
    for (int i = 0; i < grid->nx; i++) {
        free(grid->data[i]);
    }
    free(grid->data);
    grid->data = NULL;
}

int main(void) {
    omp_set_dynamic(0);
    const char *nome_csv = "resultados.csv";

    int resolucoes[] = {1024, 2048, 4096, 8192};
    int total_execucoes = sizeof(resolucoes) / sizeof(resolucoes[0]);

    for (int threads = 1; threads <= 256; threads *= 2) {
        omp_set_num_threads(threads);
        for (int i = 0; i < total_execucoes; i++) {
            int nx_atual = resolucoes[i];
            int ny_atual = resolucoes[i];

            printf("=====\n");
            printf("Iniciando simulação para o grid: %dx%d\n", nx_atual, ny_atual);

            Grid2D u_velA = alocar_grid(nx_atual, ny_atual, 1.0f);
            Grid2D u_velB = alocar_grid(nx_atual, ny_atual, 1.0f);

            Grid2D u_atual = u_velA;
            Grid2D u_proximo = u_velB;

            printf("Velocidade inicial no centro: %.2f\n",
                u_atual.data[nx_atual / 2][ny_atual / 2]);

            double tempo_inicio = omp_get_wtime();

            for (int t = 0; t < NT; t++) {
                calcular_proximo_passo(u_atual, u_proximo, VISC, DT, DX, DY);

                Grid2D temp = u_atual;
                u_atual = u_proximo;
                u_proximo = temp;
            }

            double tempo_fim = omp_get_wtime();
            double tempo_total = tempo_fim - tempo_inicio;

            printf("Velocidade final no centro após %d passos: %f\n", NT,
                u_atual.data[nx_atual / 2][ny_atual / 2]);
            printf("Tempo total gasto: %f segundos\n", tempo_total);
        }
    }
}

```

```
    salvar_metricas_csv(nome_csv, nx_atual, ny_atual, NT, tempo_total,
                       threads);

    liberar_grid(&u_velA);
    liberar_grid(&u_velB);
}
}

printf("=====\n");
printf("Todas as execuções finalizadas.\n");

return 0;
}
```